GreenBOINC: Carbon-Aware Scheduling via Volunteer Computing

Paper #130

***Abstract*—The global growth in computational demand necessitates innovative strategies to mitigate the environmental impact of computing. We introduce GreenBOINC, the first system to bring carbon-aware temporal and spatial scheduling to volunteer computing by extending BOINC, the widely used middleware that coordinates thousands of donated devices with capabilities previously limited to centralized cloud infrastructures. GreenBOINC utilizes real-time regional carbon intensity forecasts to enable a dynamic scheduler to align task execution with periods of lowest grid carbon intensity. This scheduler identifies a "Clean Window," a time interval where the carbon intensity is significantly below average, and defers task execution to that window. In addition, GreenBOINC has a Locality Shifting Scheduler that enables spatial carbon optimization by steering project assignments toward lower-emission regions (subject to client capability and community contribution targets). This mechanism uses client-reported capability, uptime, and emissions data along with a minimum cost flow allocation strategy to ensure that carbon-intensive workloads are preferentially executed in greener geographic areas. We validate this approach through emulation across grid regions, demonstrating that the system successfully identifies and utilizes low-carbon periods. Our experiments show that GreenBOINC's locality-shifting scheduler can reduce emissions by up to 15.91% while the temporal-shifting scheduler can achieve up to a 12.50% emissions reduction while maintaining 75% uptime, depending on location. Under a realistic simulation of current BOINC participation, we demonstrate that jointly applying temporal and spatial scheduling yields an emissions reduction of 18.00% at 75% uptime. These results show that adapting cloud-scale sustainability techniques in a decentralized, citizen-driven computing environment, can enable a pathway to democratize access to low-emission computation.**

I. Introduction

Volunteer computing has become one of the largest and most impactful public-resource computing ecosystems, enabling volunteers to contribute idle CPU and GPU cycles to accelerate scientific breakthroughs in astrophysics, mathematics, climate modeling, and biomedical research. Among these platforms, the Berkeley Open Infrastructure for Network Computing (BOINC) [1] is the largest, supporting around 85,000 active users worldwide [23], and serving as the foundation for the system we develop in this work. Because volunteers donate not only computation but also the electricity that powers it, carbon-aware scheduling directly benefits both the global environmental footprint of scientific computing and the individual volunteers who seek to contribute responsibly. As computing demand continues to rise, aligning volunteer computation with cleaner energy availability presents a powerful opportunity for large-scale emissions reduction.

Unlike centralized infrastructures, volunteer computing environments exhibit high variability in geographic location, creating substantially more opportunities for emissions reduction through locality shifting. They also involve diverse and heterogeneous hardware, with varying energy efficiencies and device capabilities, which constrain which machines can execute particular tasks. At the same time, volunteers expect their machines to perform useful work during the periods they have chosen to participate and often prefer to support particular scientific projects. Because users donate their hardware, they may limit usage to increase device longevity, creating opportunities to temporally shift workloads when users request lower utilization of their machines. Together, these characteristics make volunteer computing a natural and largely untapped platform for carbon-aware temporal and spatial workload scheduling.

To date, carbon-aware computing has been predominantly explored in centralized data center environments rather than in decentralized, volunteer-driven infrastructures. Data centers have become central to the global digital economy, but their energy demands are growing rapidly. In 2022, data centers consumed approximately 460 terawatt-hours (TWh) of electricity globally, and this figure is expected to more than double by 2030, reaching around 945 TWh—comparable to the entire electricity consumption of Japan [2]. In response, major cloud service providers have adopted aggressive carbon mitigation strategies. Google, for example, has implemented carbon-aware computing by shifting workloads to times and regions with cleaner energy availability, contributing to a 12% reduction in data center emissions despite a 27% increase in electricity use [3]. While these techniques have proven effective in centralized infrastructures, they have not been explored in the decentralized volunteer-computing ecosystem, which offers a complementary opportunity to displace emissions-intensive computation from data centers.

In this work, we propose *GreenBOINC*, an extension of the BOINC platform that introduces carbon-aware scheduling to volunteer computing. Our approach mirrors data center sustainability strategies by scheduling volunteer device computation based on regional carbon intensity and dynamically assigning tasks to volunteer devices. We first introduce a temporal shifting scheduler, which enables users to input a desired percent uptime, allowing the scheduler to identify low-emissions energy windows in which to execute tasks.

Beyond temporal workload shifting, GreenBOINC introduces a locality shifting scheduler that enables spatial carbon optimization across the volunteer network. BOINC projects typically offer two types of tasks: CPU tasks and GPU tasks. Through our investigations, we found that CPU tasks maintain

near-constant utilization of the processor once assigned. GPU tasks, however, exhibit substantial variation in intensity across projects, creating opportunities for GPU-based locality shifting that can reduce emissions while maintaining volunteer satisfaction and project performance. This mechanism allows high-wattage or GPU-intensive workloads to be redirected toward clients operating in lower-emission regions, extending carbon-aware scheduling beyond "when" a task runs to also include "where" it runs.

The locality shifting scheduler uses client-reported uptime, project support, and estimated emissions rates to formulate a minimum cost flow [25] task allocation problem, ensuring that computationally intensive tasks are preferentially assigned to the greenest available clients.

By combining temporal deferral with spatial task placement, GreenBOINC provides a hybrid carbon-aware strategy that leverages both diurnal variation in grid cleanliness and global diversity in volunteer energy profiles. This dual approach reflects a significant step toward bringing advanced sustainability techniques into the decentralized and heterogeneous landscape of volunteer computing.

II. Terminology and Context

In this section, we provide a summary of the underlying volunteer computing middleware, BOINC, and define the current carbon-aware terminology used to measure and implement environmental impact, and the strategy of Carbon-Aware Scheduling. We also illustrate the potential application of these techniques.

Volunteer Computing is a form of distributed computing where the general public donates the unused processing power and storage of their personal devices to support large-scale scientific or humanitarian research. It operates by breaking down massive computational tasks into small, parallel work units, which are managed by middleware and run on volunteers' machines during idle time. This approach provides researchers with tremendous, cost-effective computing capacity which can rival supercomputers [4].

BOINC is an open-source, widely adopted volunteer computing middleware that aggregates more than 85,000 [23] active volunteer resources made up of donated desktop, laptop, and servers to support large-scale scientific and engineering workloads, transforming divisible problems into small, fault-tolerant work units that run opportunistically on heterogeneous, geographically distributed devices. Its scheduler, credit system, and project-centric architecture make it resilient to node churn and varied performance characteristics while providing project teams with a cost-effective, scalable compute pool for tasks ranging from astrophysics to molecular simulations. Work units can vary between requiring one CPU thread, multiple CPU threads, or CPU threads with a GPU. These third type of work units have varying energy consumption rates which is what we exploit for our locality shifting scheduler [5].

Carbon-Aware Scheduling. Carbon-Aware Scheduling is the strategy of dynamically deferring or shifting computational tasks to execute only during periods (or in regions) where the carbon intensity is lowest, thereby minimizing the total carbon footprint of the computing activity. This process relies on obtaining carbon intensity forecasts which are geographically constrained and valid only within a specific Balancing Authority Area (Region) and using this data to calculate and define a Clean Window (or Green Window). The Clean Window is the specific time interval identified by the scheduler where the forecasted carbon intensity is significantly below average. This calculated window is then set as the sole desired time for the BOINC client to run tasks, aligning computational demand with moments of lowest grid carbon intensity.

Motivation. Because volunteers contribute energy and compute at irregular times and locations, BOINC inherently exposes temporal and spatial variability in power source mix and device availability, offering a unique platform for exploring demand-side strategies that align computation with low-carbon grid conditions. Prior work has demonstrated BOINC's ability to harness massive aggregate capacity, but it lacks native mechanisms for explicitly optimizing task placement for carbon intensity or renewable availability, creating an opportunity to extend its scheduler and client policies to minimize lifecycle emissions while preserving scientific throughput [1].

III. Related Works

Carbon-aware workload shifting has been extensively explored in centralized cloud environments. Carbon-Aware Load Balancing for Geo-Distributed Applications [6] and GreenCourier [7] demonstrate spatial carbon optimization by moving cloud workloads across regions based on real-time emissions forecasts, achieving substantial reductions by exploiting geographic variability. These systems rely on full control over the compute substrate, enabling the platform to relocate, pause, or reassign work at will.

Recent advances have broadened carbon-aware scheduling beyond cloud data centers. CarbonEdge [15] and A Carbon-Aware Scheduling Algorithm for Enhancing Energy Efficiency in Edge-Fog Computing Systems [16] explore spatial optimization at the network edge, where mesoscale carbon variation can be exploited for greener workload placement. Carbon-aware techniques have also been applied to specialized systems including machine learning clusters [9], AI inference platforms [10], and federated learning [13], [14], each leveraging temporal or spatial signals according to their architectural constraints. Although these systems incorporate different forms of carbon-aware decision-making, they share a common assumption: the computing environment is centrally managed and capable of enforcing scheduling decisions.

GreenBOINC differs fundamentally from these approaches as it must operate without the traditional control mechanisms available in cloud systems. Prior work assumes consistent access to granular carbon data, suspend/resume capabilities, workload migration, elastic scaling, or full infrastructure ownership—capabilities leveraged by systems such as CARBON-DQN [17] and EDJC [18]. These assumptions do not hold in BOINC, where hosts are intermittent, heterogeneous, autonomous, and cannot be forced to accept, migrate, or pause

work. As a result, GreenBOINC adapts carbon-aware techniques to the constraints of a decentralized, volunteer-driven ecosystem, providing a practical and democratized pathway to reducing emissions outside managed cloud infrastructures.

IV. System Design

GreenBOINC extends BOINC [1], a volunteer computing platform, by introducing emission-aware optimization. BOINC enables public-resource computing by connecting volunteers to research projects, each of which manages one or more applications through servers maintained by the project owner. Project servers distribute workunits with deadlines tailored to volunteers' characteristics, including hardware capabilities, uptime, and storage or network limits. Participants join by registering on a project website and installing the BOINC client, which can attach to multiple projects, rotate computation between them, and schedule work at specific times. When a user completes a task for a project, the project awards credit to the user based on the computation time of the task.

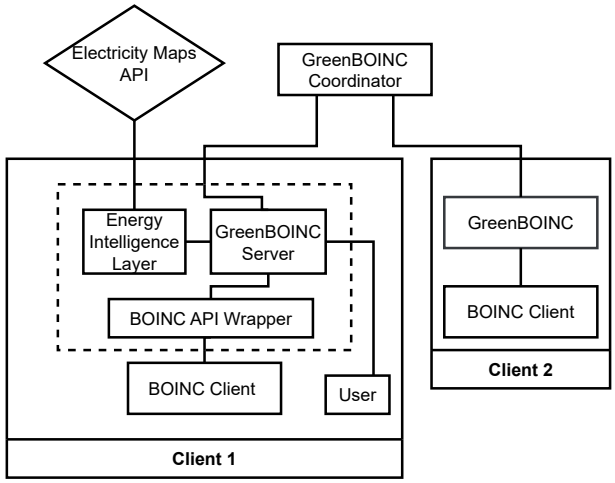


Fig. 1. GreenBOINC Architecture. The dashed box outlines GreenBOINC's internal architecture. The User interacts with the GreenBOINC server through HTML requests. GreenBOINC servers communicate with the GreenBOINC coordinator to spatially shift workloads. GreenBOINC servers communicate with the Electricity Maps API and BOINC Client to temporally shift workloads.

GreenBOINC introduces two schedulers: the Temporal Shifting Scheduler, and the Locality Shifting Scheduler. These schedulers aim to reduce emissions by scheduling computation during low-emission periods (Temporal Shifting Scheduler) and by assigning high-intensity projects to locations with cleaner energy availability (Locality Shifting Scheduler).

A. Temporal Shifting Scheduler

Our Temporal Shifting Scheduler extends the existing BOINC volunteer computing framework to prioritize execution during periods of low emissions. The system architecture integrates three major components: the GreenBOINC server, the Energy Intelligence Layer, and the BOINC API wrapper.

The server exposes REST endpoints to the user to update uptime requests and retrieve statistics. It also refreshes energy data and run time configurations every hour to keep the system up to date. The Energy Intelligence Layer retrieves carbon intensity forecasts from Electricity Maps [19], estimates hourly emissions, and identifies the cleanest windows. The third component, the BOINC API wrapper, communicates with the local volunteer BOINC client. This wrapper controls what projects the client should run as well as when they should run it.

The workflow begins when a client submits its desired uptime percentage to the server. This metric, already used in BOINC to reduce hardware wear and energy consumption, allows GreenBOINC to shift computation to low-emission periods without affecting the client's expected work contribution. The server queries the Electricity Maps API [19] through the Energy Intelligence Layer to retrieve the regional carbon intensity forecast. The API returns emission values at five-minute granularity, which we aggregate to hourly averages to provide stable scheduling intervals. The requested uptime percentage is multiplied by 24 to determine the number of hours the client should run within a day. The scheduler then chooses the lowest-emitting hours, up to the required number, to define the client's designated computation windows. Finally, the server configures the BOINC client to run only during these selected hours by updating its allowed run times. Because the BOINC client only supports setting one computation window, GreenBOINC's server runs every hour to update the client with the next window.

B. Locality Shifting Scheduler

The Locality Shifting Scheduler, running on the GreenBOINC coordinator, uses a voting system to preserve community contribution targets while minimizing emissions. Users submit votes indicating which projects they wish to support, along with desired uptime, which projects the client can run, and average emissions.

We found that GPU tasks vary in average power consumption across projects. The coordinator exploits these differences by reassigning projects across connected clients to optimize for low-emission execution, moving high-wattage tasks to low-emission regions. This approach ensures that users' preferred projects are still computed, even if their own client is not actively running them, while reducing the overall carbon footprint.

1) Voting: Users identify the projects they can run and vote for projects they want to support. Votes do not specify what an individual client must run. Instead, each client's votes express the community-level contribution targets that GreenBOINC should support in aggregate (e.g., "if GreenBOINC did not exist, this client would devote its compute to project P"). The coordinator aggregates votes across clients to derive per-project execution quotas, and then assigns those quotas to capable clients in a way that minimizes emissions. Clients indicate their capability to run projects based on their inclusion of its benchmark. If a benchmark is omitted from the request,

```
{
    user_id: 0,
    avg_emissions: 45.67,
    pct_uptime: 1,
    benchmarks:{
        Amicable Numbers: {
            Vote: 1,
            Watts: 62.4776897689769
        },
        Asteroids@Home: {
            Vote: 1,
            Watts: 85.42880718954248
        },
        Einstein@Home: {
            Vote: 1,
            Watts: 45.67930599369085
        },
        Moo! Wrapper: {
            Vote: 1,
            Watts: 34.15677100494234
        },
        PrimeGrid-AP27: {
            Vote: 1,
            Watts: 110.47581320450885
        },
        PrimeGrid-Genefer: {
            Vote: 1,
            Watts: 137.125823817292
        }
    }
}
```


Fig. 2. Example JSON Request for Locality Shifting sent from GreenBOINC servers to GreenBOINC coordinator

TABLE I
INCOMING DATA TO MCF ALGORITHM FOR FIG. 3

Clients		A	B	C
Emission Rates (gCO2/kWh)		100	350	20
Amicable Number	*Benchmark*	50W	50W	50W
	Vote	0.5	0.5	0
Einstein@Home	*Benchmark*	25W	25W	25W
	Vote	0	0	1
Asteroids@Home	*Benchmark*	75W	75W	
	Vote	0.5	0.5	

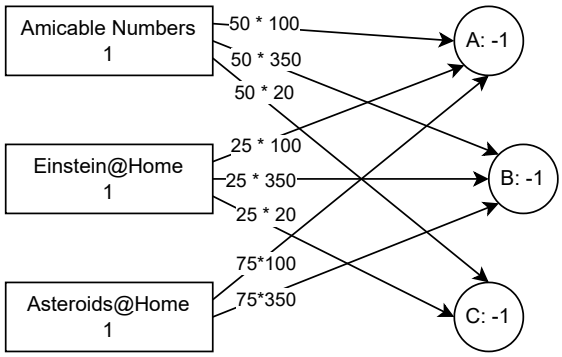


Fig. 3. Minimum Cost Flow problem setup from input data in Table I. The Amicable Numbers, Einstein@Home, and Asteroids@Home projects act as source nodes, while clients A, B, and C serve as demand nodes. Edges indicate whether a client can run a project with edge weights equal to the cost computed as the product of the client's emission rate and the client-project benchmark.

this indicates that a client cannot run that project. Along with benchmarks and votes, requests also include information about average emissions of the client as well as percent uptime. Average emissions are provided by the client to allow clients to indicate if they are running on a different power source other than their power grid. Figure 2 shows an example request servers send out. Every minute, the GreenBOINC coordinator uses the minimum cost flow [25] algorithm to calculate the lowest emitting configuration of working clients to minimize emissions while honoring all clients requested computations of tasks.

2) Task Allocation: Minimum-Cost Flow Formulation: To determine how tasks should be distributed across heterogeneous volunteer clients while minimizing total carbon emissions, we model the assignment problem as a *Minimum-Cost Flow* (MCF) optimization [25]. The MCF problem seeks the lowest-cost way to send flow through a directed network. A flow network consists of nodes with supply or demand (source and sink vertices), and directed edges that each carry a cost per unit of flow. The objective is to route flow from supply nodes to demand nodes such that all demands are satisfied and the total cost across all edges is minimized.

Consider an example. Suppose we have one supply node A with supply 2, and two demand vertices B and C with demand -1. The directed edges have costs $(A \to B)_{cost} = 1$, $(B \to C)_{cost} = 1$, and $(A \to C)_{cost} = 10$. The optimal solution sends one unit of flow directly from A to B, and one unit from A to B and then onward to C. After routing these flows, all supplies and demands are balanced, with one unit reaching B and one reaching C at minimum total cost.

In our usage of this algorithm, we will designate C as clients and P as projects. $V_{c,p}$ is the votes where c is a client and p is the project. U_c is the percent uptime, $W_{c,p}$ is the wattage per client-task pair, and E_c is the emissions. For each project we will create a supply node where the supplies of project p is equal to $\sum_{c \in C}(V_{c,p} \times U_c)$. We will also create demand nodes for each client c with demand $-U_c$. For each client c and project p, if the project is supported by the client, we will create a directed edge with unit cost $W_{c,p} * E_c$ from project p to c. Running the algorithm with this graph will create allocations of each task for each client. We use the OR-Tools Library developed by Google [26] to solve the algorithm once set up.

Consider an example using the input data in Table I. Assume all three clients (C_A, C_B, C_C) are configured for 100% uptime. Summing votes across the three clients yields supplies of 1 for Amicable Numbers (P_{AN}), 1 for Einstein@Home (P_{EH}) and 1 for Asteroids@Home (P_{AH}). With 100% uptime, each of the three sink nodes has demand -1. For each project–client pair we form an edge whose unit cost equals the project benchmark multiplied by the client's emission rate. Client C is the exception as no benchmark for Asteroids@Home has been

provided indicating that it is incapable of running this project. Therefore no edge is drawn between Asteroids@Home and Client C. The graph of this setup can be seen in Fig. 3.

The optimal flow F in this example is $F_{P_{AN}\to C_C} = 1$, $F_{P_{EH}\to C_B} = 1$, and $F_{P_{AH}\to C_A} = 1$, with all other edges carrying zero flow. Thus client A is assigned Asteroids@Home, client B is assigned Einstein@Home, and client C is assigned Amicable numbers.

We compute per-client emissions before and after optimization as follows. For client C_B the prior emissions (based on its votes) are

$$C_{B_{prior-e}} = (\sum_{p\in P} W_{C_B,p} * V_{C_B,p}) * E_{C_B} * U_{C_B} =$$

$$(50 * 0.5 + 25 * 0 + 75 * 0.5) * 350 * 1 = 21{,}875$$

and after optimization

$$C_{B_{after-e}} = (\sum_{p\in P} W_{C_B,p} * F_{p\to C_B}) * E_{C_B} * U_{C_B} =$$

$$(50 * 0 + 25 * 1 + 75 * 0) * 350 * 1 = 8{,}750$$

Doing the same calculation for other clients yield $C_{A_{prior-e}} = 6{,}250$, $C_{A_{after-e}} = 7{,}500$, $C_{C_{prior-e}} = 500$, and $C_{C_{after-e}} = 1{,}000$. Summing across clients gives total prior emissions $28{,}625$ and total after optimization emissions of $17{,}250$. The relative reduction in emissions is therefore

$$1 - 17{,}250/28{,}625 = 39.73\%$$

Note that individual clients (e.g., A and C) may see their local emissions increase if they are assigned more high-wattage work, but the global reassignment reduces total system emissions by approximately 39.73% in this example.

This formulation ensures that high-power or high-emission tasks are preferentially routed to greener clients, device heterogeneity is considered, and project-level execution proportions implied by aggregated votes are preserved.

3) Task Assignment: Due to this system design, a client can be guaranteed that when it is assigned a project different from one it explicitly voted for, the system is executing the client's voted-for projects elsewhere in the network, preserving the user's intended contribution. After GreenBOINC servers vote, they fetch their assigned projects two minutes later to allow the coordinator to calculate the task allocation. As different clients vote, assigned projects may change and therefore servers check their assigned projects every 24 hours. The system also supports new votes by clients by removing their old votes and recalculating assignments based on new votes.

V. Experiment Results

A. Setup

To perform this analysis, we used historical carbon intensity data rather than forecast data, and modified the Energy Intelligence Layer to retrieve emissions data from historical sources. Our coordinator and scheduling logic are implemented as an executable GreenBOINC server that computes run windows and locality assignments. For evaluation scale, we simulate volunteer client behavior and compute emissions from assigned project, assigned time window, benchmark power draw, and carbon-intensity traces. Thus, the scheduling decisions are produced by the real system code, while task execution and emissions accounting are simulated.

1) Temporal Shifter Setup: For our temporal shifting experiment, we emulated a volunteer client located in the California (CAISO) grid region. The purpose of this was to quantify how carbon-aware scheduling influences the total emissions associated with volunteer computation under realistic grid conditions. The carbon intensity data was obtained through the Electricity Maps [19] historical data API at a five-minute resolution over 24-hour periods and then averaged to a per hour resolution. We did this for one year of historical data stretching from 12-01-2024 to 12-01-2025 to simulate reductions over one year of processing. We varied the fraction of time selected, denoted as *F*, from 0.05 to 1.0 in order to explore the trade-off between carbon savings and task completion delay. The experiments collected several key metrics, including the average selected carbon intensity, the overall average carbon intensity and percent emission reduction. This experimental variation allowed us to study the trade-off between emissions reduction and task latency.

2) Locality Shifter Setup: We conducted a series of locality shifting experiments by creating a Bulk Client Simulator to work with the GreenBOINC Coordinator. This simulator created 500 volunteer clients, each supporting a subset of projects, voting for preferred projects, selecting a target uptime, and reporting an associated emissions rate.

In order to simulate client data, we first needed to benchmark power consumption of six high-wattage GPU projects. We did this on a representative client PC with an AMD GPU. For each project, we executed only the GPU workload for ten minutes and used AMD's Adrenalin software [27] to measure the average power draw over this interval. These measurements served as wattage benchmarks for the client simulator and can be found in Figure 2 as the *Watts* under each project's benchmark. The simulator also incorporated additional client attributes, including supported projects, project voting preferences, uptime selection, and average regional emissions.

The Bulk Client Simulator emulated multiple clients by sending concurrent requests to the GreenBOINC coordinator and computed emissions savings by comparing the emissions produced under baseline execution (running only user-voted projects) against those produced under the coordinator's carbon-optimized task assignments. Systematically varying client attributes formed the basis of our experimental evaluation.

3) Combined Experiment Setup: Finally we conducted experiments that evaluate the interaction between the locality shifting and temporal shifting schedulers. To do this, we used historical, hourly carbon intensity data from the Electricity Maps API [19] and retrieved country-level electricity carbon intensity over a one year period. We then applied our Temporal Shifting Scheduler to calculate per-country emission rates

under different user-selected uptimes, capturing carbon savings achievable through temporal deferral across countries. The temporally adjusted emission rates were subsequently used as inputs to the Bulk Client Simulator, enabling experiments with the Locality Shifting Scheduler to evaluate the combined emission savings achieved by jointly applying our Locality Shifting and Temporal Shifting Scheduler.

B. Results

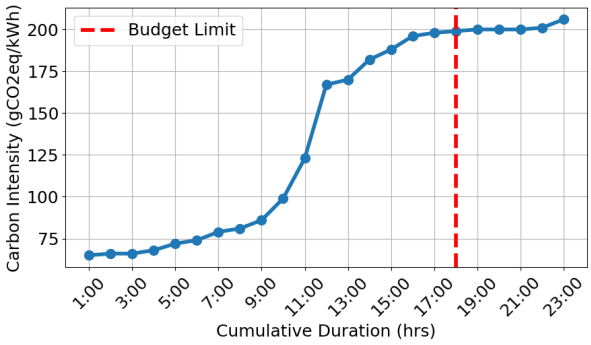


Fig. 4. Cumulative Duration vs. Carbon Intensity. Hourly emission rates are averaged and sorted from lowest to highest. The budget limit corresponds to 75% uptime, with the hours to the left of the limit selected as the selected runtime for BOINC.

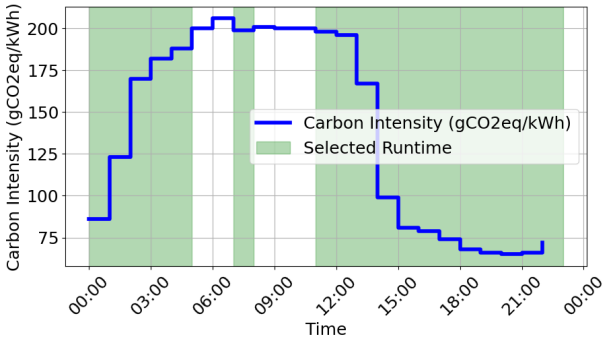


Fig. 5. Carbon Intensity Historical Data from 06-12-2025 with Selected Runtime. The blue line represents the historical carbon intensity over a 24 hour period. The lowest emitting hours over this period while maintaining 75% uptime are highlighted in green.

1) Temporal Shifter Results: We simulated GreenBOINC operating over a single day (06-12-2025) with a requested uptime of 75%. The Temporal Shifter retrieved carbon intensity data from Electricity Maps, aggregated the data into hourly intervals, and sorted these hours from lowest to highest emissions. Figure 4 illustrates the historical carbon intensity across the sorted hours. GreenBOINC then selected the lowest-emitting 75% of hours (indicated by the red line), unsorted them back into chronological order, and scheduled computation for the day accordingly, as shown in Figure 5. Compared to selecting a random 75% runtime allocation across the same day, this approach achieved a modest emissions reduction of 12.61%.

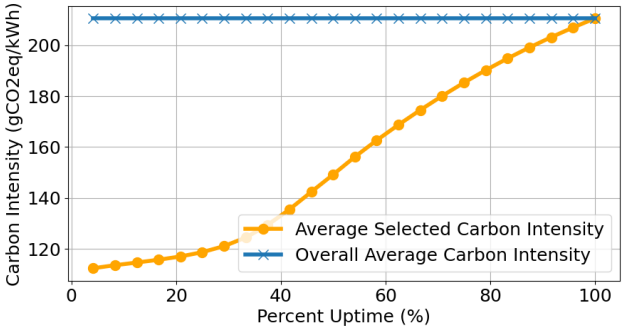


Fig. 6. Carbon Intensity vs. Percent Uptime. Average Selected Carbon Intensity indicates the emission rate over a 365-day period when GreenBOINC enforces the specified percent uptime. Overall Average Carbon intensity represents the emission rate over the same period if uptime was selected proportionally.

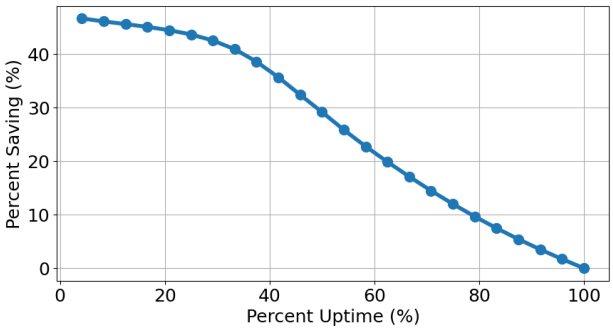


Fig. 7. Percent Saving vs Percent Uptime. The percent savings was calculated using the formula 100 × (1 - (Average Selected Carbon Intensity)/(Overall Average Carbon Intensity)) from Figure 6.

Next, we expanded the scheduler to a full-year simulation (12-01-2024 to 12-01-2025). Uptime inputs ranged from $1/24$ to $24/24$ which is 1 hour out of the 24 hour day to running 100% of the time. As a baseline, we used the overall average carbon intensity, which represents the expected emission rate if our Temporal Shifting Scheduler was not run. The difference in emission rates is shown in Fig. 6. Percent savings was computed as 100 × (1 - selected/average). The resulting savings curve is shown in Fig. 7. Running 75% of the time results in a saving rate of 11.99%, while 62.5% uptime yields 19.86% savings, and 87.5% uptime provides a saving rate of 5.42% in the CAISO grid region.

2) Locality Shifter Results: **Random Assignments:** Our initial experiments used randomly generated data to model users and evaluate how different variables influenced emissions savings. We sampled random emission values between 20 and 500 to represent the lower and upper bounds of global carbon intensity, based on ranges observed in the ElectricityMaps [19] interactive platform. All other inputs were fixed at 100% uptime with users voting equally for all projects.

Our first experiment varied the probability that a client could support a given project task, ranging from 25% to 100%. As shown in Figure 8, when clients supported tasks with only a 25% probability, the system achieved 6.92% savings, whereas

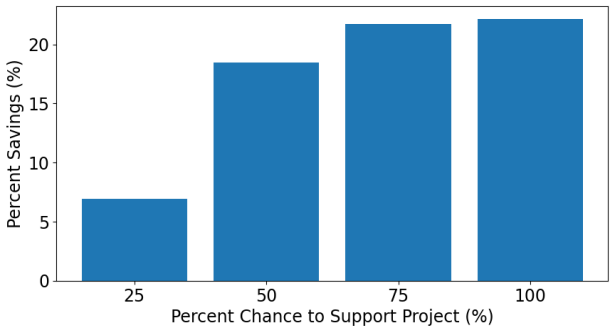


Fig. 8. Chance to Include Tasks vs. Carbon Intensity Percent Savings. Using the client simulator, we varied the probability that a client could run each task and computed the resulting percent emissions savings across 500 clients.

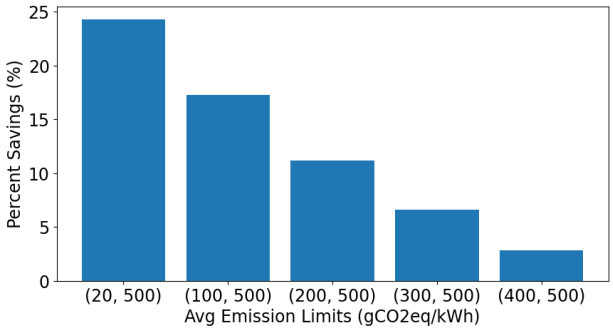


Fig. 9. Average Emission Limits vs. Carbon Intensity Percent Savings. Using the client simulator, we varied the minimum and maximum emission rates and computed the resulting percent emissions savings across 500 clients.

universal support (100%) yielded 22.14% savings. We see a sharp decrease in savings from 50% to 25% underlining that greater project support resulted in reducing emissions.

Next, we varied the emission bounds themselves, as shown in Figure 9. With wide emission bounds of $(20, 500)$, representing a large global spread, the savings rate reached 24.28%. In contrast, narrow bounds of $(400, 500)$ resulted in only 2.84% savings. This demonstrates that a larger spread in emission rates increases the opportunity to reassign tasks to substantially cleaner regions, thereby maximizing emissions reductions.

User Modeling: We then refined our user modeling to more accurately represent real BOINC usage. The first step was to construct representative emissions data, which required estimating where BOINC users are located globally. We adopted two complementary approaches: estimating the distribution of internet users per country and inferring user distribution from BOINC credit.

For the first approach, we computed the number of internet users (Column 3, Table II) in each country by multiplying the World Bank's per-capita internet usage statistics [20] by the corresponding national population values [21]. For the second approach, we leveraged self-reported country locations from the BOINCStats dataset [23], which provides live updates on BOINC credit earned by clients for completed tasks (Column 7, Table II). Because BOINC credit correlates with computational throughput, this dataset approximates the concentration of computation across countries. We used the average BOINC credit awarded over the previous seven days. Using both approaches allowed us to extrapolate potential emission savings from the existing BOINC user base to a future scenario in which volunteer computing participation scales with global internet access.

To assign emission values to users, we mapped each country to its corresponding average carbon intensity using Energy Institute data, producing the per-country emission rates shown in Column 6 of Table II.

Next, we modeled how clients vote for projects. FreeDC [24] maintains a live dashboard reporting the number of hosts connected to each BOINC project. We used these host counts as the vote weights for each project. When scaled to the modeled user population, this preserves the relative distribution of computational support across projects as observed in FreeDC's live data. We compared this to clients voting equally across all projects.

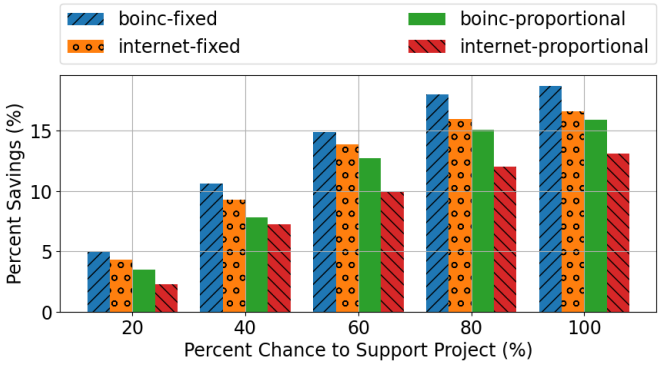


Fig. 10. Carbon Intensity Percent Savings by Percent Chance to Support Project. Using the client simulator, clients' average emissions were generated from Table II using two approaches: **internet** users per country and **boinc** credit per country, converted to carbon intensity for the corresponding country. Project support was modeled either proportionally to FreeDC host counts (**proportional**) or equally across all projects (**fixed**). The figure shows percent savings as the probability that a client can run a project is varied.

User Modeling Results: Because no dataset exists describing which computers can run which projects, we varied this capability from clients supporting 20% of projects to 100%. We evaluated four combinations of user-modeling approaches: BOINC-based country data with proportional FreeDC votes (**boinc-proportional**), BOINC-based country data with equal votes (**boinc-fixed**), internet-user–based country estimates with proportional FreeDC votes (**internet-proportional**), and internet-user–based estimates with equal votes (**internet-fixed**). Figure 10 summarizes the results. The configuration most representative of current BOINC usage (boinc–proportional) achieved a 15.91% savings when all clients supported all projects, decreasing to 15.05% at 80% support and 12.75% at 60% support. Using internet-based country estimates (internet-proportional) reduced savings to 13.07% at 100% support due to lower variance in emis-

TABLE II
SAMPLE OF INPUT CSV DATA FOR CLIENT GENERATION (SORTED DESCENDING BY BOINC CREDIT)

Country	Population	Internet Users	Percent Using Internet	Region	Intensity	BOINC Weekly Credit
United States	341963408	318519764	93.1444	North America	383.5492	6435479354
Germany	84119100	78651358	93.5	Europe	344.13992	2561961134
Canada	38794813	36450209	93.9564	North America	174.81425	1426868057
Australia	26768598	25982431	97.0631	Australia and Oceania	551.58984	1422178634
United Kingdom	68459055	65925248	96.2988	Europe	210.89114	782466865
Poland	38746310	34323418	88.585	Europe	614.98236	780180412
France	68374591	60616673	88.6538	Europe	44.179085	630741505
Czechia	10837890	9503507	87.6878	Europe	413.85922	595638368
Taiwan	23595274	22818187	96.7066	East and Southeast Asia	635.65106	572854696

sion rates. Conversely, using equal votes increased savings to 16.61% for internet-user data (internet-fixed) and 18.69% for BOINC-based data (boinc-proportional), driven by larger variance in project power consumption. Even when clients could support only 20% of projects, savings remained between 2.28% (internet-proportional) and 4.97% (boinc-fixed). Overall, across all modeling approaches, the GreenBOINC Locality Shifting Scheduler consistently reduced emissions, ranging from 9.90% (60% support with internet–proportional) up to 18.69% (100% support with boinc–fixed).

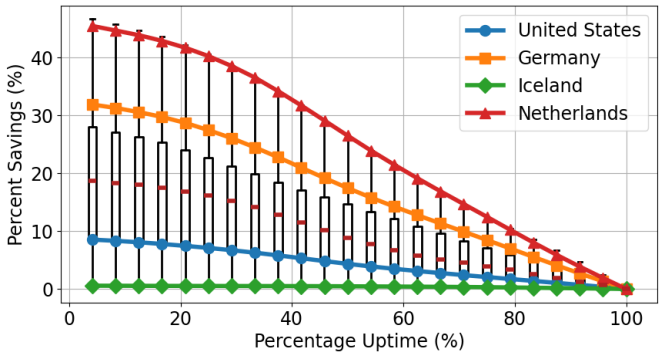


Fig. 11. Distribution of Percent Emissions Saved in Countries vs Percentage Uptime. Boxplots at each uptime percentage summarize distribution of percent-savings across different countries. Overlaid lines show four example countries' emissions saved.

3) Combined Experiment Results: To combine our temporal and spatial scheduling results, we first estimated country-specific emission rates for various countries under our temporal shifting scheduler. We used the ElectricityMaps API [19] to retrieve hourly carbon intensity data over a one year period (12-01-2024 to 12-01-2025) for a set of countries. Because our user population would be generated using BOINC Credit per country (Column 7 in Table II), we restricted this analysis for this set of countries. Each region has distinct electricity generation mixes, ranging from coal and natural gas to solar and nuclear, which leads to substantial variation in the effectiveness of temporally shifting workloads across locations. This variability is visualized in Fig. 11. The two largest BOINC Credit earning countries, the United States and Germany, exhibit pecent savings of 2.06% and 8.46% at 75% uptime respectively. Across all evaluated countries, the median at 75% uptime was 3.94% while the maximum and minimum

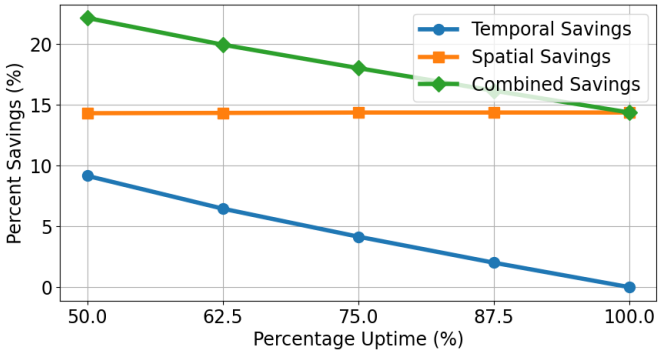


Fig. 12. Percent Emissions Savings vs Percentage Uptime for the Combined Experiment. **Temporal Savings** shows savings from the temporal-shifting scheduler; **Spatial Savings** shows savings from the locality-shifting scheduler; **Combined Savings** shows savings when both schedulers are applied. The experiment used 500 clients from the BOINC client simulator; BOINC credits per country were used to model country participation and FreeDC host counts were used to model project support. Simulations were run over one year of historical data.

observed savings were Netherlands at 12.47% and Iceland at 0.31%. This variation demonstrates the strong dependence of temporal shifting benefits on regional grid characteristics and selected uptime.

Using this generated data, we created 500 users in the Bulk Client Simulator and ran experiments with selected uptimes varying from 50% to 100%. We modified the JSON request shown in Fig. 2 by changing the "pct_uptime" field to the selected uptime. We also generated the assigned countries for users using BOINC Weekly Credit. Then, with this uptime and the user's assigned country, used the temporal shifter to calculate their emission rate and assigned it to the "avg_emissions" field. We setup all clients to support all projects and proportionally assigned votes using FreeDC host counts [24]. How we assigned countries and vote counts mirrors the **boinc-proportional** experiment with 100% chance to support projects used in the locality shifting experiments (Fig. 10).

The results from these simulated runs are shown in Fig. 12. Lower selected uptimes yield greater temporal flexibility, allowing the scheduler to defer execution away from high-emission hours and thereby increase overall savings. When clients select a 50% uptime, total emissions savings reach 22.12%, while at 75% uptime, savings are 18.00%. At 100%

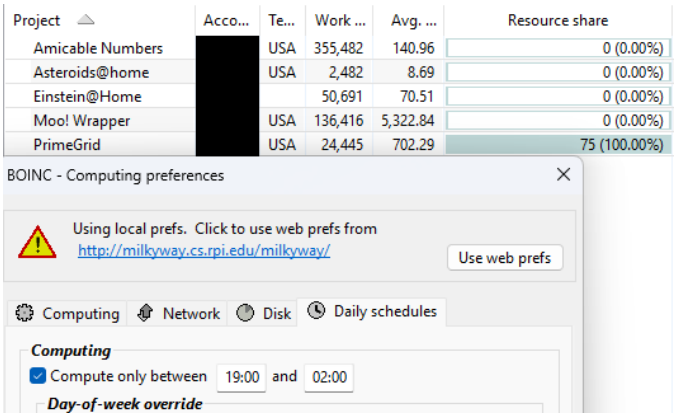


Fig. 13. Assigned Resource Share and Computing Window configurations in BOINC. The GreenBOINC server changes the *Compute only between* times to the next identified clean window. After the GreenBOINC server receives the assigned projects from the coordinator, it changes the resource share for each project. In this case, it was assigned PrimeGrid at 100%

uptime, the observed savings decrease to 14.35%, reflecting savings due solely to locality shifting. Note that this savings rate is slightly lower than the savings rate calculated in the previous **boinc-proportional** experiment due to the different input emission data we used for this experiment. These results confirm that shifting workloads temporally and spatially can generate savings rates between 14.35% and 22.12% depending on volunteer uptime preferences.

VI. End To End Timing Evaluation

To evaluate the system end to end, we deployed the GreenBOINC coordinator and instantiated 5000 simulated clients using the Bulk Client Simulator. These clients were generated using the same configuration parameters as the previous *combined* experiment. In addition, we attached a single end-to-end GreenBOINC client, running on a physical machine, to measure system timing and latency across the full workflow.

Once the GreenBOINC server is started, it queries the Electricity Maps API to retrieve forecast carbon intensity data and computes the next Clean Window while respecting the configured uptime constraint. It configures this in BOINC through restricting computing times as seen in Fig. 13. This process completes in **0.38** seconds. Next, a vote is submitted to the server and forwarded to the GreenBOINC coordinator, which periodically recomputes project assignments. The server waits two minutes for the coordinator to complete reassignment, noting that the coordinator executes this optimization once per minute. With 5,000 clients, recalculating assignments took **2.13** seconds. The one and two minute timing intervals were selected arbitrarily; even if reassignment computation were to exceed one minute at larger scales, increasing these intervals would have a negligible impact on overall emissions savings, as scheduling decisions operate at a much coarser temporal granularity.

After receiving updated project assignments, the server updates the resource shares for all voted projects and triggers synchronization requests to the local BOINC client, a process that takes **3.99** seconds. The assigned resource shares can be seen in Fig 13.

Beyond these steps, BOINC manages task execution autonomously. After the Clean Window is set as the allowed computation interval, BOINC is responsible for starting and suspending tasks accordingly. Once resource shares are assigned, BOINC rotates computation across projects in proportion to these shares. When requesting new tasks, project servers account for each client's resource share and available computation time to assign deadlines, ensuring that clients complete tasks without violating project constraints.

VII. Limitations

In our experimental evaluation, we assume that sub-projects within a single BOINC project can be selected programmatically (e.g., distinguishing between PrimeGrid Genefer and PrimeGrid AP27). This assumption enables fine-grained locality shifting across GPU workloads with differing power characteristics. In current BOINC deployments, sub-projects must be selected through the project's web interface and selecting resource share between sub-projects is not possible. Support from project maintainers and BOINC would be required to enable this capability in practice.

Another omission in our current experimental setup is the lack of consideration for volunteer heterogeneity beyond simple location and uptime. While GreenBOINC employs locality shifting to optimize for regional carbon intensity, it currently treats all volunteers as having uniform computational capability which leads to low-performance and high-performance devices contribute the same vote strength. This may distort project-level work distributions and cause divergence from the workload users intended to contribute in the absence of locality shifting.

VIII. Future Works

There are several avenues to enhance GreenBOINC's carbon-aware scheduling and further reduce emissions.

Account managers (AMs) offer an additional mechanism for carbon-aware participation. In BOINC, volunteers attach their clients to an AM account rather than directly to individual projects. The AM periodically instructs the client which project accounts to attach to and fetch jobs from. GreenBOINC could leverage this level of indirection by integrating carbon-aware policies at the AM level, allowing the AM to guide clients toward projects or workunits that minimize emissions based on real-time carbon intensity data. This would provide a centralized, volunteer-facing mechanism to influence task scheduling while preserving the autonomous nature of individual BOINC clients.

Projects could implement emission-aware workunit assignment, distributing tasks to clients based on their local emission rates while maximizing compute time. High-intensity tasks could be preferentially assigned to clients in low-carbon regions or during clean windows, further reducing overall emissions without compromising project throughput.

IX. Conclusion

In this work, we introduced GreenBOINC, a carbon-aware extension to the BOINC volunteer computing ecosystem that integrates both temporal and spatial scheduling to reduce the emissions associated with distributed scientific computation. By leveraging real-time grid carbon intensity forecasts, flexible workunit deadlines, and geographic diversity across volunteer devices, GreenBOINC demonstrates that meaningful emissions reductions are achievable without diminishing volunteer autonomy or project throughput. Our temporal shifter aligns computation with low-carbon periods, while our locality shifter assigns high-wattage GPU tasks to cleaner regions using minimum-cost flow optimization. Through experiments combining real workload characteristics, realistic emissions distributions, and user-driven project preferences, we found that GreenBOINC's Temporal Shifting Scheduler can reduce emission rates up to 12.50% at 75% uptime based on location, while the Locality Shifting Scheduler achieves a reduction of 15.91%. Combining these schedulers in a realistic simulation of current BOINC users resulted in a reduction of 18.00%. These results highlight the unique advantages of volunteer computing. As computing demand continues to grow, integrating emissions intelligence into volunteer computing frameworks represents a promising path toward more sustainable distributed computation.

Acknowledgment

The authors acknowledge the use of AI-assisted tools, including ChatGPT, Google Gemini, and Grammarly, for editing support, grammar refinement, and document preparation. All conceptual contributions, system design decisions, experimental work, and technical writing were authored and verified by the research team.

References

[1] D. P. Anderson. 2004. BOINC: A System for Public-Resource Computing and Storage. In *Proceedings of the Fifth IEEE/ACM International Workshop on Grid Computing*, pp. 4–10. https://doi.org/10.1109/GRID.2004.14

[2] International Energy Agency, "AI is set to drive surging electricity demand from data centres while offering the potential to transform how the energy sector works," 2023. [Online]. Available: https://www.iea.org/news/ai-is-set-to-drive-surging-electricity-demand-from-data-centres-while-offering-the-potential-to-transform-how-the-energy-sector-works

[3] Data Center Frontier, "Google shifting server workloads to use more renewable energy," 2023. [Online]. Available: https://www.datacenterfrontier.com/cloud/article/11429047/google-shifting-server-workloads-to-use-more-renewable-energy

[4] CERN LHC@home Project, "About volunteer computing," 2024. [Online]. Available: https://lhcathome.web.cern.ch/about/volunteer-computing

[5] University of California, Berkeley, "BOINC publications," 2024. [Online]. Available: https://boinc.berkeley.edu/pubs.php

[6] Diptyaroop Maji, Ben Pfaff, Vipin P R, Rajagopal Sreenivasan, Victor Firoiu, Sreeram Iyer, Colleen Josephson, Zhelong Pan, and Ramesh K. Sitaraman. 2023. Carbon-Aware Load Balancing for Geo-Distributed Applications. In Proceedings of the ACM SIGCOMM 2023 Conference. ACM, New York, NY, USA. https://doi.org/10.1145/3604930.3605711

[7] Mohak Chadha, Thandayuthapani Subramanian, Eishi Arima, Michael Gerndt, Martin Schulz, and Osama Abboud. 2023. GreenCourier: Carbon-Aware Scheduling for Serverless Functions. In Proceedings of the 9th International Workshop on Serverless Computing (WoSC '23). Association for Computing Machinery, New York, NY, USA, 18–23. https://doi.org/10.1145/3631295.3631396

[8] Diandian Gu, Yihao Zhao, Peng Sun, Xin Jin, and Xuanzhe Liu. 2025. GreenFlow: A Carbon-Efficient Scheduler for Deep Learning Workloads. IEEE Trans. Parallel Distrib. Syst. 36, 2 (Feb. 2025), 168–184. https://doi.org/10.1109/TPDS.2024.3470074

[9] K. Xu, D. Sun, H. Tian, J. Zhang, and K. Chen. 2025. GREEN: Carbon-efficient Resource Scheduling for Machine Learning Clusters. Proc. of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI '25).

[10] Y. Li, Z. Hu, E. Choukse, R. Fonseca, G. E. Suh, and U. Gupta. 2025. EcoServe: Designing Carbon-Aware AI Inference Systems. arXiv preprint arXiv:2502.05043.

[11] E. Rodrigues, J. Goldverg, and T. Kosar. 2025. Carbon-Aware Temporal Data Transfer Scheduling Across Cloud Datacenters. arXiv preprint arXiv:2506.04117.

[12] M. Raymond. Carbon-aware Data Pipelines: Scheduling and Optimizing Big Data Workflows in Renewable-powered Cloud Environments.

[13] D. R. Arputharaj, C. Rodriguez, A. Rodio, and G. Neglia. 2025. Green Federated Learning via Carbon-Aware Client and Time Slot Scheduling. arXiv preprint arXiv:2509.08980.

[14] J. Chen et al. 2025. GreenFL: Carbon-efficient Federated Learning over RE Powered Edge Computing Systems. ResearchGate Conference Paper.

[15] A. Souza, et al. 2025. CarbonEdge: Leveraging Mesoscale Spatial Carbon-Intensity Variations for Low Carbon Edge Computing. arXiv preprint arXiv:2502.14076.

[16] J. Sukanya. 2025. A Carbon-Aware Scheduling Algorithm for Enhancing Energy Efficiency in Edge-Fog Computing System. International Journal of Sustainable Advanced Technology (IJSAT).

[17] M. Alex, S. Ojo, F. Awuour. 2025. CARBON-DQN: Carbon-Aware, Energy-Efficient, and SLA-Compliant Virtual Machine Placement in Cloud Data Centers Using Deep Q-Networks and Agglomerative Clustering. MDPI Applied Sciences, 14(7): 280.

[18] Y. Chen, L. Luo, D. Guo, Q. He, 2025. Carbon-Aware Energy Cost Optimization of Data Analytics Across Geo-Distributed Data Centers. Journal of Computer Science and Technology, 40(3): 654-670.

[19] ElectricityMaps, "ElectricityMaps — Real-Time Electricity Mix." [Online]. Available: https://app.electricitymaps.com. [Accessed: 3-Dec-2025].

[20] World Bank, "Individuals using the Internet (% of population) — Data," [Online]. Available: https://data.worldbank.org/indicator/IT.NET.USER.ZS. [Accessed: 3-Dec-2025].

[21] CIA, "The World Factbook — Population: Country Comparison." [Online]. Available: https://www.cia.gov/the-world-factbook/field/population/country-comparison/. [Accessed: 3-Dec-2025].

[22] Ember, *Carbon intensity of electricity generation – Ember* [dataset], 2025. Energy Institute, *Statistical Review of World Energy*, 2025. Processed by Our World in Data. Sources: Ember (2025); Energy Institute (2025).

[23] BOINCstats, "BOINCstats – Country List Statistics." [Online]. Available: https://boincstats.com/stats/-1/country/list. [Accessed: 3-Dec-2025].

[24] Free-DC Stats, "Free-DC — Project Statistics." [Online]. Available: https://stats.free-dc.org/projmain. [Accessed: 4-Dec-2025].

[25] D. R. Ford and D. R. Fulkerson. 2010. Flows in Networks. Princeton University Press, USA.

[26] Google, "OR-Tools: Operations Research Tools." [Online]. Available: https://developers.google.com/optimization/. [Accessed: 3-Dec-2025].

[27] AMD, "AMD Software: Adrenalin Edition 25.11.1 Release Notes." [Online], https://www.amd.com/en/resources/support-articles/release-notes/RN-RAD-WIN-25-11-1.html/. [accessed Dec. 10, 2025].